Assignment-18.4

2303A51304

Batch-05

Task-01:

Prompt:

Write a Python script that connects to a public Movie Database API (OMDb API) using the requests library.

Requirements:

Accept movie title as user input

Fetch movie details from the API

Display:

Title, Release Year,IMDb Rating,Plot Summary

Implement proper error handling for:

Invalid movie name

API key errors

Network timeout or connection issues

Empty API response

Use clean formatting, clear comments, and readable output

Code:

```
nment-17.3 > task-01.py > ...
import requests
def fetch_movie_details(movie_title, api_key):
    url = f"http://www.omdbapi.com/?t={movie_title}&apikey={api_key}"
    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status()  # Check for HTTP errors
        data = response.json()

        if data.get("Response") == "False":
            print(f"Error: {data.get('Error')}")
            return

        print(f"Title: {data.get('Title')}")
        print(f"Release Year: {data.get('Year')}")
        print(f"IMDb Rating: {data.get('imdbRating')}")
        print(f"Plot Summary: {data.get('Plot')}")

    except requests.exceptions.HTTPError as http_err:
        print(f"HTTP error occurred: {http_err}")
    except requests.exceptions.ConnectionError as conn_err:
        print(f"Connection error occurred: {conn_err}")
    except requests.exceptions.Timeout as timeout_err:
        print(f"Timeout error occurred: {timeout_err}")
    except Exception as err:
        print(f"An error occurred: {err}")
if __name__ == "__main__":
    api_key = input("Enter your OMDb API key: ")
    movie_title = input("Enter the movie title: ")
    fetch_movie_details(movie_title, api_key)
```

Output:

```
C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>python Assignment-17.3/task-01.py
Enter your OMDb API key: fd530dbe
Enter the movie title: Inception
Title: Inception
Release Year: 2010
IMDb Rating: 8.8
Plot Summary: A thief who steals corporate secrets through the use of dream-sharing technology is given the inverse task of planting an idea i
nto the mind of a CEO, but his tragic past may doom the project and his team to disaster.
```

Explanation:

The program uses the requests library to connect to the OMDb Movie API and fetch movie details from the internet.
It builds a URL using the movie title and API key entered by the user and sends a request to the API.
The response is converted into JSON format and the program extracts title, year, rating, and plot information.
If the API returns an error (invalid movie or key), a message is shown instead of crashing.

Exception handling manages HTTP errors, connection problems, and timeouts to ensure the program runs safely.

Task-02:

Prompt:

Write a Python script that integrates a Public Transport Schedule API using the requests library.

Requirements:

Accept station/stop ID as user input

Validate that the stop ID is not empty and is alphanumeric

Fetch arrival timing data from a transport API endpoint

Display the next 3 arrival times in a clean formatted output

Error Handling:

Handle invalid stop ID responses from API

Handle server unavailable errors (5xx status codes)

Handle network timeout and connection errors

Implement retry logic with at least 1 automatic retry if request fails

Implementation Details:

Use try-except blocks for exception handling

Use requests.get() with timeout

Show user-friendly error messages instead of crashes

Code:

```python
import requests
import time

# ---- Mock API function (simulates real API response) ----
def mock_api_request(stop_id):
    """Simulated API response"""
    mock_data = {
        "101": {"arrival_times": ["10:15 AM", "10:25 AM", "10:40 AM", "11:05 AM"]},
        "202": {"arrival_times": ["09:50 AM", "10:05 AM", "10:30 AM"]},
        "303": {"arrival_times": ["11:00 AM", "11:20 AM", "11:45 AM"]}
    }
    return mock_data.get(stop_id, {})


# ---- Fetch arrival times with retry logic ----
def fetch_arrival_times(stop_id):
    retries = 1   # minimum one retry

    while retries >= 0:
        try:
            # Simulating API request (replace with real API later)
            response_data = mock_api_request(stop_id)

            # Invalid stop ID check
            if not response_data or "arrival_times" not in response_data:
                print("Error: Invalid stop ID or no arrival data found.")
                return

            # Display next 3 arrivals
            arrival_times = response_data["arrival_times"][:3]

            print("\nUpcoming Arrivals")
            print("------------------")
            for arrival in arrival_times:
                print(f"• {arrival}")
```

```python
def fetch_arrival_times(stop_id):
            # Display next 3 arrivals
            arrival_times = response_data["arrival_times"][:3]

            print("\nUpcoming Arrivals")
            print("-------------------")
            for arrival in arrival_times:
                print(f"• {arrival}")

            return

        except requests.exceptions.Timeout:
            print("Timeout error occurred. Retrying...")
            retries -= 1
            time.sleep(2)

        except requests.exceptions.ConnectionError:
            print("Connection error occurred. Retrying...")
            retries -= 1
            time.sleep(2)

        except Exception as err:
            print(f"An unexpected error occurred: {err}")
            return


# ---- Main Program ----
if __name__ == "__main__":
    stop_id = input("Enter the station/stop ID: ").strip()

    # Input validation
    if not stop_id or not stop_id.isalnum():
        print("Error: Stop ID must be alphanumeric and cannot be empty.")
    else:
        fetch_arrival_times(stop_id)
```

Output:

```
Enter the station/stop ID: 101

Upcoming Arrivals
-------------------
• 10:15 AM
• 10:25 AM
• 10:40 AM
```

Explanation:

The program accepts a station or stop ID from the user and validates that the input is not empty and is alphanumeric.
It simulates a transport API request using mock data to retrieve arrival timings for the given stop.
Retry logic is implemented so the program attempts the request again if a timeout or connection error occurs.
The next three arrival times are extracted and displayed in a clean formatted output.
Exception handling using try-except blocks prevents crashes and shows clear error messages for invalid inputs or failures.

Task-03:

Prompt:

Write a Python script that fetches live stock market data using a financial API (such as Alpha Vantage or any public stock API) with the requests library.

Requirements:

Accept stock symbol as user input (example: AAPL, TSLA)

Send API request to retrieve stock data

Display:

Current price

Day high price

Day low price

Error Handling:

Show clear message for invalid stock symbol

Handle API rate limit responses

Handle missing or incomplete fields in JSON response

Validate API response before parsing JSON

Prevent program crashes using try-except blocks

Implementation Details:

Use requests.get() with timeout

Safely parse JSON using checks before accessing keys

Format output cleanly and clearly

Write readable, well-commented code runnable in VS Code

Assume API returns JSON stock data.

Code:

```python
import requests
def fetch_stock_data(stock_symbol, api_key):
    url = f"https://www.alphavantage.co/query?function=GLOBAL_QUOTE&symbol={stock_symbol}&apikey={api_key}"
    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status()  # Check for HTTP errors
        data = response.json()

        if "Global Quote" not in data or not data["Global Quote"]:
            print("Error: Invalid stock symbol or no data found.")
            return

        quote = data["Global Quote"]
        current_price = quote.get("05. price", "N/A")
        day_high = quote.get("03. high", "N/A")
        day_low = quote.get("04. low", "N/A")

        print(f"Stock Symbol: {stock_symbol.upper()}")
        print(f"Current Price: {current_price}")
        print(f"Day High: {day_high}")
        print(f"Day Low: {day_low}")

    except requests.exceptions.HTTPError as http_err:
        if response.status_code == 429:
            print("Error: API rate limit exceeded. Please try again later.")
        else:
            print(f"HTTP error occurred: {http_err}")
    except requests.exceptions.ConnectionError as conn_err:
        print(f"Connection error occurred: {conn_err}")
    except requests.exceptions.Timeout as timeout_err:
        print(f"Timeout error occurred: {timeout_err}")
    except Exception as err:
        print(f"An error occurred: {err}")
if __name__ == "__main__":
    api_key = input("Enter your Alpha Vantage API key: ")
    stock_symbol = input("Enter the stock symbol (e.g., AAPL, TSLA): ")
    fetch_stock_data(stock_symbol, api_key)
```

Output:

```
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding> & C:/Use
eDrive/Desktop/AI_Assistance_coding/Assignment-18.4/task-03.py
Enter your Alpha Vantage API key: xxxx
Enter the stock symbol (e.g., AAPL, TSLA): AAPL
Stock Symbol: AAPL
Current Price: 252.6200
Day High: 255.0000
Day Low: 251.6000
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
```

Explanation:

The program takes a stock symbol and an API key from the user to request live stock data from the Alpha Vantage financial API.
It sends an HTTP request using the requests library and converts the API response into JSON format.
The script safely extracts the current price, day high, and day low values using dictionary checks to avoid missing-field errors.
Error handling with try-except blocks manages invalid symbols, network issues, timeouts, and API rate limits without crashing.
Finally, the stock information is printed in a clean and readable format for the user.

Task-04:

Prompt:

Write a Python script that integrates a Geolocation API using the requests library to detect the geographical location of a user based on an IP address.

Requirements:

Accept an IP address as user input

Validate the IP address format before making the API request

Fetch and display:

Country

City

ISP (Internet Service Provider)

Error Handling:

Show an error message for invalid IP address format

Handle API request failures or server errors

Handle JSON decoding errors safely

Prevent crashes using try-except blocks

Implementation Details:

Use requests.get() with timeout

Validate input using Python's ipaddress module

Safely parse JSON response with key checks

Display output in a clean formatted structure

Write readable, well-commented Python code runnable in VS Code

Assume API returns JSON containing country, city, and isp fields.

Code:

ignment-18.4 > task-04.py > fetch_geolocation

```python
import requests
import ipaddress
def fetch_geolocation(ip_address):
    url = f"https://ipapi.co/{ip_address}/json/"
    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status()  # Check for HTTP errors
        data = response.json()

        country = data.get("country_name", "N/A")
        city = data.get("city", "N/A")
        isp = data.get("org", "N/A")

        print(f"IP Address: {ip_address}")
        print(f"Country: {country}")
        print(f"City: {city}")
        print(f"ISP: {isp}")

    except requests.exceptions.HTTPError as http_err:
        print(f"HTTP error occurred: {http_err}")
    except requests.exceptions.ConnectionError as conn_err:
        print(f"Connection error occurred: {conn_err}")
    except requests.exceptions.Timeout as timeout_err:
        print(f"Timeout error occurred: {timeout_err}")
    except ValueError as json_err:
        print(f"JSON decoding error: {json_err}")
    except Exception as err:
        print(f"An error occurred: {err}")
if __name__ == "__main__":
    ip_address = input("Enter an IP address: ")
    try:
        # Validate IP address format
        ipaddress.ip_address(ip_address)
        fetch_geolocation(ip_address)
    except ValueError:
        print("Error: Invalid IP address format. Please enter a valid IP address.")
```

Output:

```
eDrive/Desktop/AI_Assistance_coding/Assignment-18.4/task-04.py
Enter an IP address: 8.8.8.8
IP Address: 8.8.8.8
Country: United States
City: Mountain View
ISP: Google LLC
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
```

Explanation:

The program takes an IP address from the user and first validates whether the format is a valid IP using built-in checks.
If the IP is valid, it sends a request to a Geolocation API using the requests library to retrieve location details.
The API response is converted into JSON and the country, city, and ISP information are extracted safely.
Error handling manages invalid IP formats, API failures, network issues, and JSON decoding problems so the program does not crash.
Finally, the geolocation details are displayed in a clean, readable output format.

Task-05:

Prompt:

Write a Python script that simulates payment processing using a sandbox Payment Gateway API with the requests library.

Requirements:

- Accept user input for:

  • Amount

  • Currency (e.g., USD, INR)

  • Payment method (card, upi, netbanking)

- Validate inputs:

- Amount must be positive number
- Currency must be alphabetic
- Payment method must not be empty

Implementation:

Send a POST request to a sandbox payment API endpoint

Pass payment details as JSON payload

Display transaction status clearly (Success / Failed)

Error Handling:

Handle invalid payment details (HTTP 400)

Handle unauthorized access (HTTP 401)

Handle server errors (HTTP 500)

Handle network interruption and timeout errors

Use try-except blocks to prevent crashes

Logging:

- Log all failed transactions into a file named "failed_transactions.log"
- Include timestamp, amount, currency, and error message

Technical Details:

Use requests.post() with timeout

Validate API response before parsing JSON

Code:

```python
import json
from datetime import datetime

# ----------- Log Failed Transactions -----------
def log_failure(amount, currency, method, error_msg):
    with open("failed_transactions.log", "a") as file:
        file.write(
            f"{datetime.now()} | Amount:{amount} | Currency:{currency} "
            f"| Method:{method} | Error:{error_msg}\n"
        )

# ----------- Simulated Payment API -----------
def simulate_payment_api(payload):
    """
    Mock sandbox API response.
    Replace this with real API URL later.
    """
    if payload["amount"] <= 0:
        return {"status_code": 400, "message": "Invalid payment details"}

    if payload["payment_method"].lower() not in ["card", "upi", "netbanking"]:
        return {"status_code": 400, "message": "Unsupported payment method"}

    # Simulated success
    return {
        "status_code": 200,
        "transaction_id": "TXN123456",
        "status": "Success"
    }

# ----------- Payment Processing -----------
def process_payment(amount, currency, method):
    payload = {
        "amount": amount,
        "currency": currency.upper(),
        "payment_method": method.lower()
    }
```

```python
def process_payment(amount, currency, method):
    payload = {
        "amount": amount,
        "currency": currency.upper(),
        "payment_method": method.lower()
    }

    try:
        # Simulated POST request (mock API)
        response = simulate_payment_api(payload)

        status_code = response["status_code"]

        # ---- Handle HTTP-like responses ----
        if status_code == 200:
            print("\nPayment Successful")
            print("-------------------")
            print(f"Transaction ID: {response['transaction_id']}")
            print(f"Status: {response['status']}")

        elif status_code == 400:
            error_msg = "Invalid payment details."
            print(f"Error: {error_msg}")
            log_failure(amount, currency, method, error_msg)

        elif status_code == 401:
            error_msg = "Unauthorized access."
            print(f"Error: {error_msg}")
            log_failure(amount, currency, method, error_msg)

        elif status_code == 500:
            error_msg = "Server error occurred."
            print(f"Error: {error_msg}")
            log_failure(amount, currency, method, error_msg)
```

```python
def process_payment(amount, currency, method):
        log_failure(amount, currency, method, error_msg)

    except requests.exceptions.ConnectionError:
        error_msg = "Network connection error."
        print(error_msg)
        log_failure(amount, currency, method, error_msg)

    except Exception as e:
        error_msg = f"Unexpected error: {e}"
        print(error_msg)
        log_failure(amount, currency, method, error_msg)

# ---------- Main Program ----------
if __name__ == "__main__":
    try:
        amount = float(input("Enter amount: "))
        currency = input("Enter currency (USD/INR): ").strip()
        method = input("Enter payment method (card/upi/netbanking): ").strip()

        # Input validation
        if amount <= 0:
            print("Amount must be greater than zero.")
        elif not currency.isalpha():
            print("Currency must contain only letters.")
        elif method == "":
            print("Payment method cannot be empty.")
        else:
            process_payment(amount, currency, method)

    except ValueError:
        print("Invalid amount entered. Please enter a numeric value.")
```

Output:

```
eDrive/Desktop/AI_Assistance_coding/Assignment-18.4/task-05.py
Enter amount: 500
Enter currency (USD/INR): USD
Enter payment method (card/upi/netbanking): card

Payment Successful
------------------
Transaction ID: TXN123456
Status: Success
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
```

Explanation:

The script collects payment details from the user. Input validation ensures amount is positive and fields are valid. A POST request is sent to a test payment API. Errors like invalid data, server issues, or

connection failures are handled safely. Failed transactions are saved in a log file for tracking.